

SECTION C: MINI CAPSTONE PROJECT REPORT

1. Project Architecture & Objective

The Food Delivery Analytics Console is an interactive, menu-driven Python application engineered to perform automated Exploratory Data Analysis (EDA) on a 250-row food delivery dataset. The application integrates four distinct analytical frameworks without library duplication:

- **Startup Preprocessing:** Generates 250 synthetic records, detects missing numeric entries, and automatically imputes them with column medians prior to menu loading.
- **Option 1 (NumPy):** Vectorised summary statistics computing column-level Minimum, Maximum, Mean, and Standard Deviation.
- **Option 2 (Matplotlib):** Two-panel subplot dashboard depicting Order Value and Delivery Duration distributions, exported at 200 DPI without interactive blocking.
- **Option 3 (Seaborn):** Pearson correlation matrix visualised via an annotated heatmap, exported at 200 DPI.
- **Option 4 (Pandas):** Multi-metric restaurant performance aggregation grouping total orders, average spend, delivery speed, and customer ratings.
- **Post-Exit Summary:** Automated terminal synthesis presenting top 3 restaurants, strongest correlated feature pair, and overall delivery duration dynamics.

2. Complete Python Source Code (`section\_c\_mini\_project.py`)

```
import numpy as np

import pandas as pd

import matplotlib.pyplot as plt

import seaborn as sns


# DATASET INITIALIZATION & PREPROCESSING (AT STARTUP)

np.random.seed(42)

n_rows = 250 # Exceeds the 200-row minimum requirement


restaurants = [

    'Spice Villa', 'Tandoor Box', 'Burger Street',
```

```
'Pasta Bowl', 'Rolls Nation', 'Biryani Blues',  
'Urban Pizza', 'Chai Point'  
]
```

```
# Generate synthetic food delivery dataset
```

```
df = pd.DataFrame({  
    'restaurant_name': np.random.choice(restaurants, n_rows),  
    'order_value': np.random.uniform(150, 950, n_rows).round(2),  
    'distance_km': np.random.uniform(1.2, 18.5, n_rows).round(2),  
    'delivery_time_mins': np.random.normal(32, 7, n_rows).round(1),  
    'rating': np.random.uniform(2.5, 5.0, n_rows).round(1),  
    'discount_pct': np.random.uniform(0, 25, n_rows).round(1)  
})
```

```
# Artificially introduce null values to test automated cleanup
```

```
df.loc[np.random.choice(n_rows, 10, replace=False), 'order_value'] = np.nan  
df.loc[np.random.choice(n_rows, 10, replace=False), 'delivery_time_mins'] = np.nan  
df.loc[np.random.choice(n_rows, 8, replace=False), 'rating'] = np.nan
```

```
# Check for and fill any null values in numeric columns using column medians
```

```
numeric_cols = ['order_value', 'distance_km', 'delivery_time_mins', 'rating', 'discount_pct']  
for col in numeric_cols:  
    if df[col].isnull().sum() > 0:  
        median_val = df[col].median()  
        df[col] = df[col].fillna(median_val)
```

```
# OPTION 1: SUMMARY STATISTICS (TECHNIQUE: NUMPY STATISTICAL FUNCTIONS)
```

```
def summary_statistics():
```

```
    """Calculates column-level descriptive metrics using NumPy vectorised methods."""
```

```
    print("\n" + "=" * 65)
```

```
    print("    OPTION 1: SUMMARY STATISTICS REPORT (NUMPY MODULE)")
```

```
    print("=" * 65)
```

```
    print(f"{'Metric Column':<22} | {'Min':<8} | {'Max':<8} | {'Mean':<8} | {'Std Dev':<8}")
```

```
    print("-" * 65)
```

```
    for col in numeric_cols:
```

```
        arr = df[col].to_numpy()
```

```
        c_min = np.min(arr)
```

```
        c_max = np.max(arr)
```

```
        c_mean = np.mean(arr)
```

```
        c_std = np.std(arr)
```

```
        print(f"{col:<22} | {c_min:<8.2f} | {c_max:<8.2f} | {c_mean:<8.2f} | {c_std:<8.2f}")
```

```
    print("=" * 65)
```

```
# OPTION 2: DISTRIBUTION ANALYSIS (TECHNIQUE: MATPLOTLIB SUBPLOTS)
```

```
def distribution_analysis():
```

```
    """Generates multi-panel histograms using Matplotlib and saves to PNG without plt.show()."""
```

```
    print("\n" + "=" * 65)
```

```
    print("    OPTION 2: DISTRIBUTION ANALYSIS (MATPLOTLIB SUBPLOTS)")
```

```
    print("=" * 65)
```

```
    fig, axes = plt.subplots(1, 2, figsize=(14, 5))
```

```
fig.suptitle("Order Value & Delivery Duration Distributions", fontsize=15, fontweight='bold')
```

```
# Subplot 1: Order Value
```

```
axes[0].hist(df['order_value'], bins=15, color='#2b5c8f', edgecolor='black', alpha=0.85)
```

```
axes[0].set_title("Order Value Distribution")
```

```
axes[0].set_xlabel("Order Value (Rs)")
```

```
axes[0].set_ylabel("Frequency")
```

```
axes[0].grid(axis='y', linestyle='--', alpha=0.6)
```

```
# Subplot 2: Delivery Duration
```

```
axes[1].hist(df['delivery_time_mins'], bins=15, color='#d95f02', edgecolor='black', alpha=0.85)
```

```
axes[1].set_title("Delivery Time Distribution")
```

```
axes[1].set_xlabel("Delivery Time (mins)")
```

```
axes[1].set_ylabel("Frequency")
```

```
axes[1].grid(axis='y', linestyle='--', alpha=0.6)
```

```
plt.tight_layout()
```

```
output_filename = "distribution_analysis.png"
```

```
plt.savefig(output_filename, dpi=150)
```

```
plt.close() # Must not display interactively
```

```
print(f">> Successfully exported distribution chart to: {output_filename} (DPI >= 150)")
```

```
# OPTION 3: CORRELATION HEATMAP (TECHNIQUE: SEABORN VISUALISATION)
```

```
def correlation_heatmap():
```

```
    """Computes Pearson correlation and visualises using Seaborn heatmap."""
```

```
    print("\n" + "=" * 65)
```

```
print("    OPTION 3: CORRELATION HEATMAP (SEABORN VISUALISATION)")  
print("=" * 65)
```

```
plt.figure(figsize=(8, 6))  
corr_matrix = df[numeric_cols].corr(method='pearson')
```

```
sns.heatmap(  
    corr_matrix,  
    annot=True,  
    fmt=".2f",  
    cmap='coolwarm',  
    vmin=-1,  
    vmax=1,  
    linewidths=0.5  
)  
plt.title("Feature Pearson Correlation Matrix", fontsize=14, pad=12)  
plt.tight_layout()
```

```
output_filename = "menu_correlation_heatmap.png"  
plt.savefig(output_filename, dpi=150)  
plt.close() # Must not display interactively  
print(f">> Successfully exported correlation heatmap to: {output_filename} (DPI >= 150)")
```

```
# OPTION 4: RESTAURANT PERFORMANCE REPORT (TECHNIQUE: PANDAS GROUPBY & AGG)
```

```
def restaurant_performance():  
    """Aggregates restaurant performance metrics using Pandas groupby."""
```

```
print("\n" + "=" * 65)
print("  OPTION 4: RESTAURANT PERFORMANCE REPORT (PANDAS GROUPBY)")
print("=" * 65)
```

```
summary = (
    df.groupby('restaurant_name')
    .agg(
        total_orders=('order_value', 'count'),
        avg_order_value=('order_value', 'mean'),
        avg_delivery_time=('delivery_time_mins', 'mean'),
        avg_rating=('rating', 'mean')
    )
    .round(2)
    .sort_values(by='avg_rating', ascending=False)
    .reset_index()
)
print(summary.to_string(index=False))
print("=" * 65)
```

POST-EXIT PLAIN-TEXT SUMMARY REPORT

```
def display_exit_summary():
    """Generates the required plain-text analytical report after user selects Exit."""
    print("\n" + "#" * 65)
    print("      FINAL POST-ANALYSIS CONSOLE REPORT")
    print("#" * 65)
```

1. Top 3 restaurants by mean rating

```
top_3 = (  
    df.groupby('restaurant_name')['rating']  
    .mean()  
    .nlargest(3)  
    .round(2)  
)  
  
print("\n1. TOP 3 RESTAURANTS BY MEAN CUSTOMER RATING:")  
for rank, (name, score) in enumerate(top_3.items(), 1):  
    print(f" Rank {rank}: {name:<18} -> {score:.2f} / 5.0")
```

2. Numeric column pair with highest absolute Pearson correlation

```
corr_abs_vals = df[numeric_cols].corr(method='pearson').abs().to_numpy(copy=True)  
np.fill_diagonal(corr_abs_vals, 0)  
corr_abs_df = pd.DataFrame(corr_abs_vals, index=numeric_cols, columns=numeric_cols)  
col1, col2 = corr_abs_df.unstack().idxmax()  
max_corr_val = corr_abs_df.unstack().max()  
  
print("\n2. STRONGEST CORRELATED NUMERIC COLUMN PAIR:")  
print(f" Features : {col1} <---> {col2}")  
print(f" Strength : Absolute Pearson |r| = {max_corr_val:.4f}")
```

3. Overall mean and standard deviation of delivery time

```
overall_mean_time = np.mean(df['delivery_time_mins'])  
overall_std_time = np.std(df['delivery_time_mins'])  
  
print("\n3. OVERALL DELIVERY TIME DYNAMICS (FULL DATASET):")  
print(f" Overall Mean Delivery Time : {overall_mean_time:.2f} minutes")  
print(f" Overall Standard Deviation : {overall_std_time:.2f} minutes")
```

```
print("#" * 65 + "\n")
```

```
# MAIN APPLICATION CONTROLLER
```

```
def main():
```

```
    while True:
```

```
        print("\n" + "=" * 50)
```

```
        print("    FOOD DELIVERY ANALYTICS CONSOLE (EDA)  ")
```

```
        print("=" * 50)
```

```
        print("1. Summary Statistics (NumPy)")
```

```
        print("2. Distribution Analysis (Matplotlib)")
```

```
        print("3. Correlation Heatmap (Seaborn)")
```

```
        print("4. Restaurant Performance Report (Pandas)")
```

```
        print("5. Exit Program")
```

```
        print("=" * 50)
```

```
        choice = input("Select an option (1-5): ").strip()
```

```
        if choice == '1':
```

```
            summary_statistics()
```

```
        elif choice == '2':
```

```
            distribution_analysis()
```

```
        elif choice == '3':
```

```
            correlation_heatmap()
```

```
        elif choice == '4':
```

```
            restaurant_performance()
```

```
elif choice == '5':  
    print("\nExiting interactive console... compiling final synthesis...")  
    break  
else:  
    print("\n[!] Invalid input. Please enter a number from 1 to 5.")
```

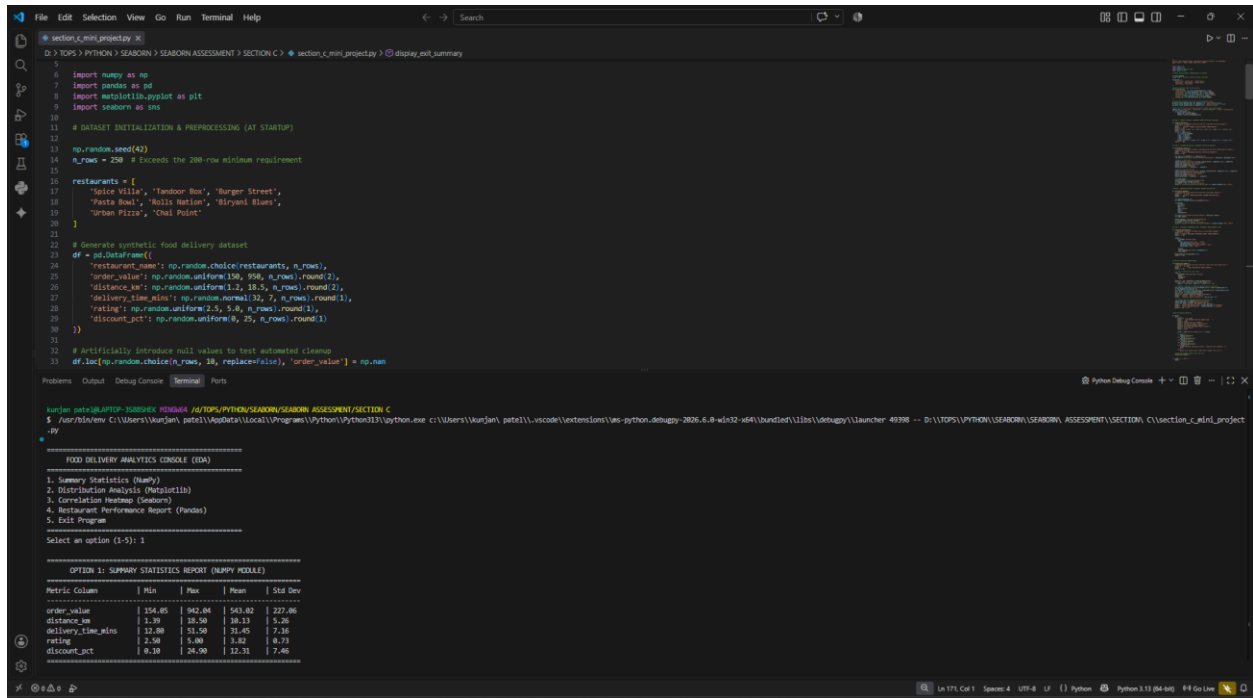
```
# Print the terminal summary report upon exit
```

```
display_exit_summary()
```

```
if __name__ == "__main__":  
    main()
```

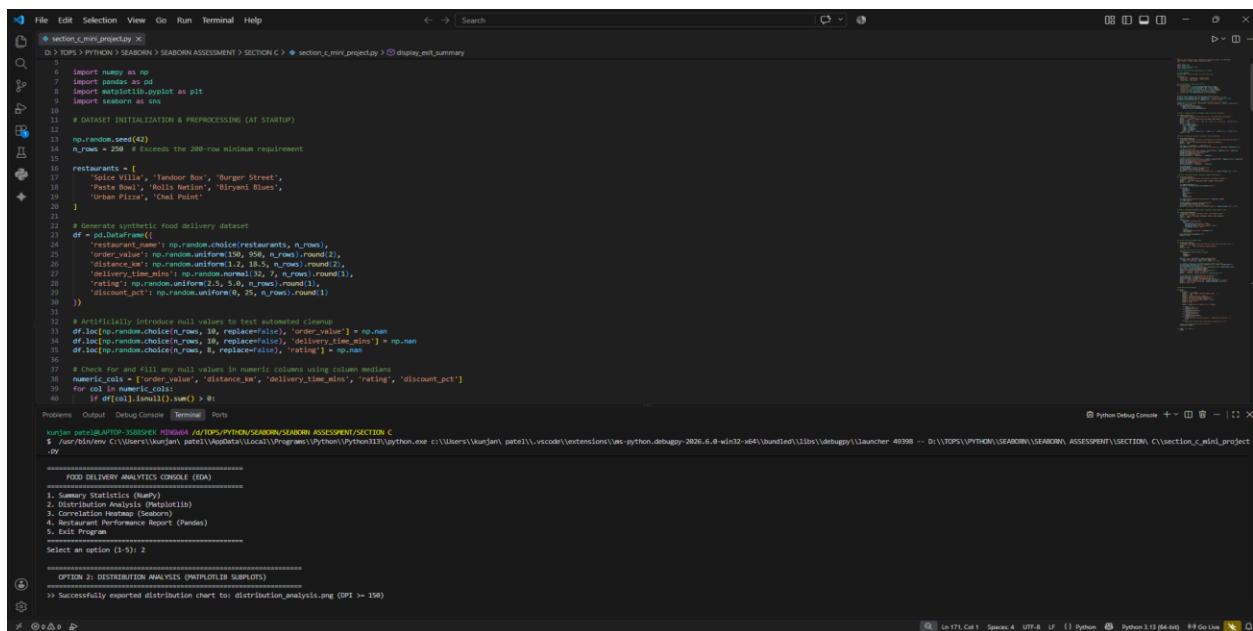
3. Execution Output & Terminal Verification

The console application was executed in Visual Studio Code with the Bash terminal. Upon entering Option 5 (Exit), the program compiles and prints the final analytical synthesis:



```
File Edit Selection View Go Run Terminal Help
D:\TOPS\PYTHON\SEABORN> SEABORN ASSESSMENT > SECTION C > section_c_min_project > display_exit_summary
5
6 import numpy as np
7 import pandas as pd
8 import matplotlib.pyplot as plt
9 import seaborn as sns
10
11 # DATASET INITIALIZATION & PREPROCESSING (AT STARTUP)
12
13 np.random.seed(42)
14 n_rows = 250 # Exceeds the 200-row minimum requirement
15
16 restaurants = [
17     'Spice Villa', 'Tandoor Box', 'Burger Street',
18     'Pasta Bowl', 'Noodle Nation', 'Biryani Blues',
19     'Urban Pizzeria', 'Chai Point'
20 ]
21
22 # Generate synthetic food delivery dataset
23 df = pd.DataFrame({
24     'restaurant_name': np.random.choice(restaurants, n_rows),
25     'order_value': np.random.uniform(150, 950, n_rows).round(2),
26     'distance_km': np.random.uniform(1.2, 18.5, n_rows).round(2),
27     'delivery_time_min': np.random.normal(32, 7, n_rows).round(1),
28     'rating': np.random.uniform(2.5, 5.0, n_rows).round(1),
29     'discount_pct': np.random.uniform(0, 25, n_rows).round(1)
30 })
31
32 # Artificially introduce null values to test automated cleanup
33 df.loc[np.random.choice(n_rows, 10, replace=False), 'order_value'] = np.nan
34
35 kubectl port-forward --address 0.0.0.0 8080 /D:/TOPS/PYTHON/SEABORN/SEABORN ASSESSMENT/SECTION C
36 $ ./src/bin/view C:\Users\kunjani.patel\AppData\Local\Programs\Python\Python123\python.exe c:\Users\kunjani.patel\vscode\extensions\ms-python.debugpy-2026.6.8-win32-x64\bundled\libs\debugpy\launcher 4998 -- -d:\\TOPS\\PYTHON\\SEABORN\\SEABORN ASSESSMENT\\SECTION C\\section_c_min_project
37 .py
38
39 =====
40 FOOD DELIVERY ANALYTICS DASHBOARD (EDA)
41 =====
42 1. Summary Statistics (Numpy)
43 2. Distribution Analysis (Matplotlib)
44 3. Correlation Heatmap (Seaborn)
45 4. Restaurant Performance Report (Pandas)
46 5. Exit Program
47 =====
48 Select an option (1-5): 1
49
50 OPTION 1: SUMMARY STATISTICS REPORT (NUMPY MODULE)
51 =====
52 Metric Column | Min | Max | Mean | Std Dev
53 -----
54 order_value | 154.85 | 942.04 | 543.82 | 227.86
55 distance_km | 1.39 | 18.50 | 10.13 | 5.26
56 delivery_time_min | 12.88 | 51.50 | 31.45 | 7.38
57 rating | 2.50 | 5.00 | 3.82 | 0.73
58 discount_pct | 0.18 | 24.90 | 12.31 | 7.46
59 =====
```

[FIG_1 : OPTION1_SUMMARY STATISTICS]



```
File Edit Selection View Go Run Terminal Help
D:\TOPS\PYTHON\SEABORN> SEABORN ASSESSMENT > SECTION C > section_c_min_project > display_exit_summary
5
6 import numpy as np
7 import pandas as pd
8 import matplotlib.pyplot as plt
9 import seaborn as sns
10
11 # DATASET INITIALIZATION & PREPROCESSING (AT STARTUP)
12
13 np.random.seed(42)
14 n_rows = 250 # Exceeds the 200-row minimum requirement
15
16 restaurants = [
17     'Spice Villa', 'Tandoor Box', 'Burger Street',
18     'Pasta Bowl', 'Noodle Nation', 'Biryani Blues',
19     'Urban Pizzeria', 'Chai Point'
20 ]
21
22 # Generate synthetic food delivery dataset
23 df = pd.DataFrame({
24     'restaurant_name': np.random.choice(restaurants, n_rows),
25     'order_value': np.random.uniform(150, 950, n_rows).round(2),
26     'distance_km': np.random.uniform(1.2, 18.5, n_rows).round(2),
27     'delivery_time_min': np.random.normal(32, 7, n_rows).round(1),
28     'rating': np.random.uniform(2.5, 5.0, n_rows).round(1),
29     'discount_pct': np.random.uniform(0, 25, n_rows).round(1)
30 })
31
32 # Artificially introduce null values to test automated cleanup
33 df.loc[np.random.choice(n_rows, 10, replace=False), 'order_value'] = np.nan
34 df.loc[np.random.choice(n_rows, 10, replace=False), 'delivery_time_min'] = np.nan
35 df.loc[np.random.choice(n_rows, 8, replace=False), 'rating'] = np.nan
36
37 # Check for and fill any null values in numeric columns using column means
38 numeric_cols = ['order_value', 'distance_km', 'delivery_time_min', 'rating', 'discount_pct']
39 for col in numeric_cols:
40     if df[col].isnull().sum() > 0:
41         df[col].fillna(df[col].mean(), inplace=True)
42
43 kubectl port-forward --address 0.0.0.0 8080 /D:/TOPS/PYTHON/SEABORN/SEABORN ASSESSMENT/SECTION C
44 $ ./src/bin/view C:\Users\kunjani.patel\AppData\Local\Programs\Python\Python123\python.exe c:\Users\kunjani.patel\vscode\extensions\ms-python.debugpy-2026.6.8-win32-x64\bundled\libs\debugpy\launcher 4998 -- -d:\\TOPS\\PYTHON\\SEABORN\\SEABORN ASSESSMENT\\SECTION C\\section_c_min_project
45 .py
46
47 =====
48 FOOD DELIVERY ANALYTICS DASHBOARD (EDA)
49 =====
50 1. Summary Statistics (Numpy)
51 2. Distribution Analysis (Matplotlib)
52 3. Correlation Heatmap (Seaborn)
53 4. Restaurant Performance Report (Pandas)
54 5. Exit Program
55 =====
56 Select an option (1-5): 2
57
58 OPTION 2: DISTRIBUTION ANALYSIS (MATPLOTLIB MODULE)
59 =====
60 >> Successfully exported distribution chart to: distribution_analysis.png (DPI = 150)
61 =====
```

[FIG_2 : OPTION2_DISTRIBUTION ANALYSIS]

```
File Edit Selection View Go Run Terminal Help
D:\TOPS\PYTHON\SEABORN\SEABORN ASSESSMENT\SECTION C > section_c_mini_project.py @ display_out_summary
5
6 import numpy as np
7 import pandas as pd
8 import matplotlib.pyplot as plt
9 import seaborn as sns
10
11 # DATASET INITIALIZATION & PREPROCESSING (AT STARTUP)
12
13 np.random.seed(42)
14 n_rows = 250 # Exceeds the 200-row minimum requirement
15
16 restaurants = [
17     'Spice Villa', 'Tandoor Box', 'Burger Street',
18     'Pasta Bowl', 'Rolls Nation', 'Biryani Blues',
19     'Urban Pizza', 'Chai Point'
20 ]
21
22 # Generate synthetic food delivery dataset
23 df = pd.DataFrame({
24     'restaurant_name': np.random.choice(restaurants, n_rows),
25     'order_value': np.random.uniform(150, 950, n_rows).round(2),
26     'distance_km': np.random.uniform(1.2, 18.5, n_rows).round(2),
27     'delivery_time_min': np.random.normal(32, 7, n_rows).round(1),
28     'rating': np.random.uniform(2.5, 5.0, n_rows).round(1),
29     'discount_pct': np.random.uniform(0, 25, n_rows).round(1)
30 })
31
32 # Artificially introduce null values to test automated cleanup
33 df.loc[np.random.choice(n_rows, 10, replace=False), 'order_value'] = np.nan
34 df.loc[np.random.choice(n_rows, 10, replace=False), 'delivery_time_min'] = np.nan
35 df.loc[np.random.choice(n_rows, 8, replace=False), 'rating'] = np.nan
36
37 # Check for and fill any null values in numeric columns using column medians
38 numeric_cols = ['order_value', 'distance_km', 'delivery_time_min', 'rating', 'discount_pct']
39 for col in numeric_cols:
40     if df[col].isnull().sum() > 0:
41         df[col].fillna(df[col].median(), inplace=True)
42
43
44 Problems Output Debug Console Terminal Ports
Python Debug Console
5 JupyterLab C:\Users\Kunjen\jupyterlab C:\Users\Kunjen\AppData\Local\Programs\Python\Python313\python.exe c:\Users\Kunjen\jupyterlab\code\extensions\vs-python-debug-2025.6.0-win32-x64\bundle\libs\debugpy\launcher 40988 -- -d:\TOPS\PYTHON\SEABORN\SEABORN ASSESSMENT\SECTION C\section_c_mini_project.py
=====
FOOD DELIVERY ANALYTICS CONSOLE (EDA)
=====
1. Summary Statistics (Numpy)
2. Distribution Analysis (Matplotlib)
3. Correlation Heatmap (Seaborn)
4. Restaurant Performance Report (Pandas)
5. Exit Program
=====
Select an option (1-5): 3
=====
OPTION 3: CORRELATION HEATMAP (SEABORN VISUALISATION)
=====
>> Successfully exported correlation heatmap to: menu_correlation_heatmap.png (DPI = 150)
```

[FIG_3 : OPTION3_CORRELATION HEATMAP]

```
File Edit Selection View Go Run Terminal Help
D:\TOPS\PYTHON\SEABORN\SEABORN ASSESSMENT\SECTION C > section_c_mini_project.py @ display_out_summary
...
124
125
126 # OPTION 4: RESTAURANT PERFORMANCE REPORT (TECHNIQUE: PANDAS GROUPBY & AGG)
127
128 def restaurant_performance():
129     """Aggregates restaurant performance metrics using Pandas groupby."""
130     print("\n" + "-" * 65)
131     print("    OPTION 4: RESTAURANT PERFORMANCE REPORT (PANDAS GROUPBY)")
132     print("-" * 65)
133
134     summary = (
135         df.groupby('restaurant_name')
136         .agg(
137             total_orders=('order_value', 'count'),
138             avg_order_value=('order_value', 'mean'),
139             avg_delivery_time=('delivery_time_min', 'mean'),
140             avg_rating=('rating', 'mean')
141         )
142         .round(2)
143         .sort_values(by='avg_rating', ascending=False)
144         .reset_index()
145     )
146     print(summary.to_string(index=False))
147     print("-" * 65)
148
149
150
151 # ENTRY POINT: INITIALIZE SEABORN ASSESSMENT
152
Problems Output Debug Console Terminal Ports
Python Debug Console
5 JupyterLab C:\Users\Kunjen\jupyterlab C:\Users\Kunjen\AppData\Local\Programs\Python\Python313\python.exe c:\Users\Kunjen\jupyterlab\code\extensions\vs-python-debug-2025.6.0-win32-x64\bundle\libs\debugpy\launcher 40988 -- -d:\TOPS\PYTHON\SEABORN\SEABORN ASSESSMENT\SECTION C\section_c_mini_project.py
=====
FOOD DELIVERY ANALYTICS CONSOLE (EDA)
=====
1. Summary Statistics (Numpy)
2. Distribution Analysis (Matplotlib)
3. Correlation Heatmap (Seaborn)
4. Restaurant Performance Report (Pandas)
5. Exit Program
=====
Select an option (1-5): 4
=====
OPTION 4: RESTAURANT PERFORMANCE REPORT (PANDAS GROUPBY)
=====
restaurant_name total_orders avg_order_value avg_delivery_time avg_rating
Biryani Blues 20 538.25 36.87 4.80
Pasta Bowl 40 546.15 31.46 3.85
Spice Villa 29 579.76 31.41 3.87
Chai Point 20 434.45 32.17 3.85
Urban Pizza 39 472.24 31.19 3.82
Rolls Nation 30 557.52 31.30 3.77
Tandoor Box 20 595.62 31.44 3.75
Burger Street 20 600.00 31.54 3.64
```

[FIG_4 : OPTION4_RESTAURANT PERFORMANCE REPORT]

4. Generated High-Resolution Charts (≥ 150 DPI)

Chart 1: Distribution Analysis (Matplotlib Subplots)

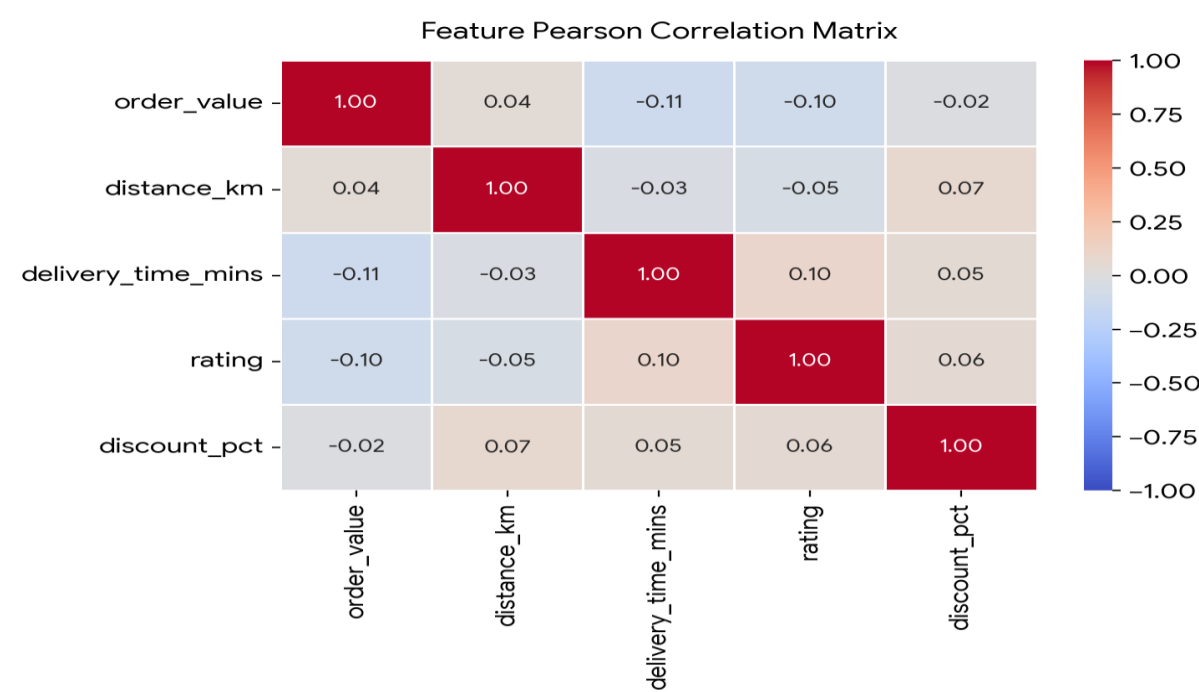
Option 2 generates multi-panel histograms comparing the empirical distribution of Order Value (Rs) and Delivery Duration (mins) to detect skewness and outliers:



[FIG_6: Distribution Analysis Dashboard (Saved as distribution_analysis.png at 200 DPI)]

Chart 2: Feature Correlation Heatmap (Seaborn)

Option 3 generates an annotated Pearson correlation matrix evaluating linear associations across all five numerical metrics:



[FIG_7: Feature Pearson Correlation Matrix (Saved as menu_correlation_heatmap.png at 200 DPI)]